

Relatório Técnico — Checkpoint 1: Núcleo de Rede

Projeto: BitTorrent P2P Híbrido

Disciplina: Programação Distribuída — UEMS

Checkpoint: 01 — Network

Data de entrega: 18/09/2026

Autores: Guilherme Zanan Piveta (Aluno 1) e João Vitor Antunes da Silva (Aluno 2)

1. Introdução

Este relatório descreve a implementação do primeiro checkpoint do projeto de BitTorrent P2P híbrido. O objetivo do checkpoint foi construir o núcleo mínimo de rede necessário para que dois processos distribuídos se comuniquem por TCP, com mensagens serializadas, framing, checksum, identificação de nós e registro básico em um Super Peer.

De acordo com a especificação, o Checkpoint 1 deveria contemplar: processo distribuído, socket TCP, cliente/servidor, serialização, framing de mensagens, concorrência básica e identificação de nós. A divisão de trabalho foi mantida conforme o enunciado: o Aluno 1 ficou responsável por `network.c`, `protocol.c` e `peer.c`; o Aluno 2 ficou responsável por `node.c`, configuração e `superpeer.c`.

O sistema integrado resultante estabelece uma arquitetura em que o processo `peer` se conecta a um `superpeer`, envia mensagens de controle (`PING`, `JOIN`, `LEAVE`) e recebe respostas (`PONG`, `ACK`, `ERROR`). O Super Peer aceita conexões TCP, interpreta o protocolo de comunicação, mantém uma tabela local de membros e processa as

mensagens. Essa comunicação é abstraída por chamadas intermediadas por um middleware, responsável por gerenciar a lógica de sockets, CRC32, framing, pack e unpacking de forma transparente.

2. Escopo Implementado

O escopo entregue no CP1 pode ser organizado em quatro eixos:

- transporte TCP IPv4 e I/O de stream, implementados em `src/common/network.c` e expostos por `include/common/network.h`;
- protocolo comum de mensagens, incluindo header, serialização, framing e CRC32, implementado em `src/common/protocol.c` e exposto por `include/common/protocol.h`;
- identidade e configuração dos nós, implementadas em `src/common/node.c`, `src/common/config.c` e respectivos headers;
- processos executáveis `peer` e `superpeer`, implementados em `src/peer/peer.c`, `src/superpeer/superpeer.c` e `src/superpeer/main.c`.

Ficaram deliberadamente fora deste checkpoint os mecanismos de outros checkpoints como Chord, Gossip/heartbeat periódico, eleição Bully, State Machine Replication, 2PC, IST, upload/download real, LZ4, cache LFU e pipeline de chunks. Entretanto, para evitar retrabalho futuro, alguns tipos e campos já aparecem no protocolo, mesmo que seus respectivos handlers serão tratados apenas nos próximos checkpoints.

3. Arquitetura da Solução

A implementação segue uma separação em camadas. A camada de rede abstrai os sockets POSIX e oferece primitivas de conexão e transferência completa de bytes. Acima dela, a camada de protocolo define o formato das mensagens e garante que o receptor leia exatamente um header e o payload associado, validando tamanho,

versão, tipo e checksum. Ambas ficam em um diretório comum, pois são necessárias tanto para `peer` quanto para `superpeer`. Por fim, os processos de aplicação usam essas APIs para exercer os casos de uso do checkpoint.

Fluxo geral de uma mensagem `JOIN`:

1. O `peer` carrega sua configuração, gera seu `NodeID` e abre uma conexão TCP com o Super Peer.
2. O `peer` monta um payload `JOIN` e chama a API do protocolo para enviar header + payload.
3. O `superpeer` aceita a conexão, chama `recv_message` e recebe a mensagem já validada e desserializada.
4. O handler do Super Peer valida o `JOIN`, atualiza a tabela de membros e responde `ACK`; em caso de erro, responde `ERROR`.
5. O `peer` lê a resposta, imprime o tipo recebido e encerra a conexão.

Essa divisão evitou que o Super Peer reimplementasse socket, framing, endianness ou CRC32. O servidor e o cliente trabalham no nível da aplicação, enquanto `network.c` e `protocol.c` concentram as responsabilidades de transporte.

4. Transporte TCP e I/O de Stream

A camada de rede foi implementada em `src/common/network.c`. Ela encapsula as chamadas POSIX `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv` e `close`, disponibilizando uma API pequena para o restante do projeto:

- `net_listen`: cria um socket TCP passivo, aplica `SO_REUSEADDR`, faz `bind` em `INADDR_ANY` e entra em `listen`;
- `net_connect`: conecta um cliente a um endereço IPv4 e porta;
- `net_accept`: aceita uma conexão pendente e devolve o endereço remoto;
- `net_close`: fecha o descritor de socket;
- `send_all`: repete `send` até enviar todos os bytes solicitados;
- `recv_all`: repete `recv` até ler todos os bytes solicitados,

distinguindo EOF (`NET_CLOSED`) de erro (`NET_ERROR`).

Uma decisão importante foi tratar `send_all` e `recv_all` apenas como I/O de stream. Elas não sabem onde uma mensagem começa ou termina, por isso, recebem somente um tamanho de bytes para se transferir. O framing ficou em `protocol.c`, que primeiro lê o header de tamanho fixo e depois lê o payload indicado por `pl_size`.

5. Protocolo de Mensagens

O protocolo comum foi implementado em `src/common/protocol.c` e descrito em `include/common/protocol.h`. Toda mensagem possui um header fixo de 99 bytes, seguido de `pl_size` bytes de payload.

Layout do header:

Offset	Tamanho	Campo	Descrição
0	1	<code>protocol_ver</code>	versão do protocolo (<code>PROTOCOL_VER</code> = 1)
1	2	<code>msg_type</code>	tipo da mensagem em big-endian
3	32	<code>src_node</code>	NodeID da origem
35	32	<code>dst_node</code>	NodeID do destino
67	16	<code>trsc_id</code>	TransactionID de 128 bits
83	8	<code>time</code>	timestamp Unix
91	4	<code>pl_size</code>	tamanho do payload
95	4	<code>checksum</code>	CRC32 do payload

Campos multibyte são serializados em big-endian e nenhum struct C é diretamente enviado pelo socket, pois o padding e endianness nas máquinas poderiam ser diferentes. Em vez disso, `pack_header` e `unpack_header` copiam campo a campo para o buffer de rede.

Os payloads fixos do CP1 são:

- PING e PONG: sem payload;
- JOIN: 39 bytes (`node_id`, IPv4, porta e tipo do nó);
- LEAVE: 32 bytes (`node_id`);
- ACK: 32 bytes (`node_id` confirmado);
- ERROR: 68 bytes (`code` e `reason`).

O CRC32 é calculado com `zlib` sobre o payload, não sobre o header. No envio, `send_message` empacota o payload, calcula o checksum, serializa o header e envia `HEADER_SIZE + pl_size` bytes. Na recepção, `recv_message` lê o header, valida versão, tipo e tamanho, lê o payload, recomputa o CRC32 e só entrega a mensagem se a verificação passar.

Essa decisão torna a validação de integridade responsabilidade do protocolo, e não dos handlers de aplicação. Assim, mensagens com checksum inválido não são registradas na tabela de membros nem processadas como comandos válidos. Além disso, o código se torna mais extensível, dado que para implementar novos tipos de mensagem não é necessário modificações na validação, mas sim apenas em como ela deve ser representada (pack e unpack).

6. Identidade e Configuração dos Nós

A identidade de cada nó foi implementada em `src/common/node.c`. A especificação define:

```
NodeID = SHA256(IP || Porta || UUID)
```

Na implementação, essa concatenação possui comprimento fixo: 4 bytes de IPv4, 2 bytes de porta em big-endian e 16 bytes de UUID. O resultado é um `NodeID` de 32 bytes, armazenado como bytes crus em `node_id_t`. Além disso, uma representação hexadecimal é usada para exibição em logs.

O SHA-256 foi implementado como um wrapper sobre a API EVP do OpenSSL. Essa decisão evitou uma implementação própria de hash e manteve um ponto único de uso para checkpoints futuros,

especialmente para o pipeline de arquivos. O UUID é gerado lendo 16 bytes de `/dev/urandom`, sem dependência de `libuuid`.

Foi implementado um parser de configuração de arquivos `.conf`, em `src/common/config.c`, embora a especificação agrupe a responsabilidade de configuração junto de `node.c`. A separação evita misturar leitura de arquivo com identidade criptográfica e permite que `peer` e `superpeer` reutilizem o mesmo contrato. O formato aceito é `chave=valor`, com as chaves obrigatórias `ip`, `port` e `type`, além de `bootstrap` opcional.

O parser aceita comentários e espaços em torno de chave/valor, mas recusa chaves desconhecidas, duplicadas, linhas sem `=`, IP inválido, porta fora da faixa e tipo diferente de `peer` ou `superpeer`. A lista de bootstrap é lida como endpoints `ip:port`, separados por vírgula, mas ainda não é usada para entrada em overlay neste checkpoint.

7. Cliente P2P

O cliente `peer`, implementado em `src/peer/peer.c`, recebe pela CLI um comando (`ping`, `join` ou `leave`), o `host` e a porta do Super Peer. Em seguida, carrega sua configuração, gera seu `NodeID`, conecta ao servidor com `net_connect`, monta o payload correspondente e envia a mensagem por meio da camada de protocolo.

Ao receber a resposta, o cliente imprime `TX <TIPO>` para a mensagem enviada e `RX <TIPO>` para a mensagem recebida. Em caso de sucesso, `ping` recebe `PONG`, enquanto `join` e `leave` recebem `ACK`.

Exemplo de execução de um `peer` para cada tipo de comando:

```
./bin/client --cmd join --host 127.0.0.1 --port 8001
./bin/client --cmd ping --host 127.0.0.1 --port 8001
./bin/client --cmd leave --host 127.0.0.1 --port 8001
```

8. Super Peer e Tabela de Membros

O Super Peer foi implementado em `src/superpeer/superpeer.c`, com CLI em `src/superpeer/main.c`. O processo pode ser iniciado com um caminho de configuração simples ou com flags:

```
./bin/superpeer config/sp1.conf
./bin/superpeer --config tests/config/cl.conf --port 5510
```

Durante a inicialização, `superpeer_init` carrega o `.conf`, exige `type=superpeer`, gera UUID e NodeID, imprime a identificação do processo, insere o próprio Super Peer na tabela de membros como `MEMBER_ALIVE`, inicializa um mutex e abre o socket de escuta com `net_listen`.

A tabela de membros (`member_table_t`) é um array fixo em memória com capacidade definida por `MEMBER_TABLE_MAX_SIZE`. Cada entrada contém NodeID, IPv4, porta, tipo (`PEER` ou `SUPERPEER`), estado, `last_heartbeat` e `version`. No CP1, os estados efetivamente exercitados são `ALIVE` e `REMOVED`; `SUSPECT` e `FAILED` já existem para serem usados posteriormente com o Gossip.

Quando uma mensagem `JOIN` chega, o `superpeer` verifica se a existência do endereço, caso já exista, a entrada é atualizada em vez de duplicada, e a versão é incrementada, caso contrário, um novo registro é adicionado. Essa decisão evita que reinicializações com UUID novo criem múltiplas entradas para o mesmo endpoint. Caso a tabela esteja cheia e o `JOIN` seja de um novo endereço, o Super Peer responde `ERROR` com código 2 para evitar overflow.

A validação de `JOIN` verifica se `join.node_id` coincide com `src_node` do header, se o ID não é zero, se IPv4 e porta são válidos e se `node_type` é aceito. O Super Peer não atribui um novo NodeID ao peer, mas sim apenas confirma o ID calculado pelo próprio nó e o ecoa no `ACK`. Essa decisão mantém responsabilidade de identificação como uma propriedade local do participante.

9. Concorrência Básica

A concorrência do CP1 foi implementada com uma thread despachante por conexão. O loop principal de `superpeer_run` chama `net_accept` e para cada conexão aceita, cria uma worker thread que executa o processamento da mensagem. Se a criação da thread falhar, a conexão é tratada de forma síncrona para não descartar a requisição.

Como múltiplas conexões podem executar `JOIN` ou `LEAVE` simultaneamente, a tabela de membros é protegida por `members_lock`. O lock cobre a atualização da tabela e a impressão após `JOIN`, evitando interleaving de modificações. Não foi implementado pool de threads nem conexão persistente, pois o cliente do CP1 opera em modo comando/resposta, com uma mensagem por conexão.

10. Decisões Técnicas Relevantes

Algumas escolhas precisaram ser fechadas durante a implementação para reduzir ambiguidade e manter o checkpoint funcional.

1. **Separação entre transporte e protocolo.** `send_all` e `recv_all` tratam apenas bytes. Os header, payload e CRC32 ficam em `protocol.c`.
2. **Serialização explícita.** Nenhuma struct é enviada diretamente. Isso evita dependência de padding e endianness do host.
3. **CRC32 sobre o payload.** O checksum valida o payload da mensagem e simplifica o empacotamento do header.
4. **OpenSSL para SHA-256.** O projeto utiliza EVP do Openssl para evitar uma implementação manual de hash.
5. **UUID não persistido.** O `NodeID` muda a cada execução do processo. Identidade estável pode ser adicionada em checkpoint futuro.
6. **bind em todas as interfaces.** O IP da configuração é usado para identidade e anúncio, enquanto o servidor escuta em `INADDR_ANY`.

7. **Super Peer não gera o ID do joiner.** O `NodeID` é calculado localmente pelo nó que entra; o `ACK` apenas confirma o valor registrado.
8. **Upsert por IP+porta.** A tabela evita duplicações para o mesmo endpoint, mesmo que o `UUID` gere outro `NodeID` em uma nova execução.
9. **Thread por conexão.** A solução atende à concorrência básica exigida pela especificação sem antecipar uma arquitetura de pool ou sessões longas.
10. **Estados futuros preservados.** Estados de membership e tipos de mensagem de checkpoints posteriores já existem, mas sem comportamento distribuído completo no CP1.

11. Compilação e Execução

Na raiz do repositório:

```
make
```

O build gera `bin/client`, `bin/superpeer` e `bin/node`. O binário `bin/node` é uma cópia de `bin/superpeer`, usada pelo script de demonstração. As dependências externas do CP1 são `libcrypto` (SHA-256) e `libz` (CRC32).

Demonstração manual em dois terminais:

```
# Terminal 1
./bin/superpeer config/sp1.conf

# Terminal 2
./bin/client --cmd join --host 127.0.0.1 --port 8001
./bin/client --cmd ping --host 127.0.0.1 --port 8001
./bin/client --cmd leave --host 127.0.0.1 --port 8001
```

Demonstração automatizada:

```
bash scripts/demo/script_testes_cp1.sh
```

12. Verificação dos Requisitos do CP1

Os itens de verificação definidos na especificação foram atendidos da seguinte forma:

Item	Evidência técnica
Conexão TCP funcionando	peer usa net_connect; superpeer usa net_listen e net_accept.
Mensagem chega corretamente	send_message e recv_message transportam header + payload; JOIN registra membro.
Header é interpretado	pack_header e unpack_header validam versão, tipo, origem, destino, transação, timestamp, tamanho e checksum.
Checksum é validado	CRC32 divergente faz recv_message retornar erro e impede o processamento da mensagem.
Dois processos conversam	client recebe RX ACK para JOIN/LEAVE e RX PONG para PING.
Sem segmentation fault	Entradas inválidas, tipo não suportado, tabela cheia e desconexão são tratados com erro controlado.

Além disso, os requisitos específicos do Aluno 2 foram cobertos por node_id_generate, node_config_load, identificação no stdout, member_table_*, superpeer_handle_join e inicialização do Super Peer.

13. Testes Automatizados

Foram usados testes unitários e um script de integração.

Testes unitários:

```
make test
```

Cobertura principal:

- `tests/common/test_node.c`: vetores conhecidos de SHA-256, determinismo, efeito de IP/porta/UUID sobre o `NodeID`, comparação lexicográfica e conversão para hex;
- `tests/common/test_config.c`: arquivos `.conf` válidos e inválidos, bootstrap, campos obrigatórios, comentários e espaços;
- `tests/superpeer/test_superpeer.c`: tabela de membros, upsert, busca, tabela cheia, JOIN válido, repetido, malformado e com tipo inválido;
- `tests/c1/test_protocol.c`: tamanho do header, versão, tipos de controle, round-trip de header e tamanhos de payload no fio.

O script `scripts/demo/script_testes_cp1.sh` fornecido pelo professor executa a verificação ponta a ponta: compila os binários, sobe um Super Peer, roda os testes de protocolo, executa PING, JOIN, LEAVE, dispara 20 clientes concorrentes, confere o log do servidor e valida rejeição de mensagem com versão inválida.

13.1 Evidência de Execução

Execução realizada em 18/09/2026:

```
$ make test
bin/test_node
42 testes ok
bin/test_config
45 testes ok
bin/test_superpeer
51 testes ok
```

```
$ cd tests/c1 && make && ./test_protocol
20 testes ok

$ bash scripts/demo/script_testes_cp1.sh
=== CHECKPOINT C1 – TCP / PROTOCOLO / FRAMING / CRC ===
[PASS] compilado
20 testes ok
[PASS] Testes unitários do protocolo
[PASS] Servidor TCP aceita conexões
[PASS] PING → PONG
[PASS] JOIN → ACK
[PASS] LEAVE → ACK
[PASS] 20 clientes concorrentes
[PASS] Framing/processamento de mensagens no servidor
[PASS] Rejeição de versão de protocolo inválida
[PASS] Inicialização e identificação do nó

=== RESULTADO C1 ===
PASS: 10
FAIL: 0
CHECKPOINT C1: APROVADO
```

14. Conclusão

O Checkpoint 1 entregou o núcleo de comunicação necessário para a evolução do projeto. A implementação permite que processos `peer` e `superpeer` se comuniquem por TCP usando mensagens padronizadas, com serialização explícita, framing, CRC32 e identificação dos nós. O Super Peer mantém uma tabela básica de membros, processa `JOIN`, `LEAVE` e `PING`, e atende à exigência de concorrência básica por meio de threads por conexão.

As decisões tomadas priorizaram modularidade e extensibilidade: sockets, protocolo, identidade, configuração e membership foram isolados em módulos próprios, evitando duplicação de responsabilidades. Com isso, os próximos checkpoints podem avançar sobre uma base de rede fundamentada.